



DesignerX

Command Manual

Design skills and anti-pattern detection for any AI coding agent.
27 commands · 18 agent platforms · one terminal detector.

Crafted by Daniel Rodrigues

Version 1.0.0 · Apache License 2.0

Derived from Impeccable by Paul Bakaus. See NOTICE.md.

1. Quick start

DesignerX gives your AI coding agent a design brain: a skill with 27 commands, a library of design references, and a terminal detector that flags common UI anti-patterns. It works in Claude Code, Cursor, Gemini CLI, OpenAI Codex, GitHub Copilot, OpenCode and 12 more agents.

Install

From a project folder (npm package, once published):

```
npx designerx install --providers=claude,cursor
```

Or from a clone of the repository:

```
git clone <your-designerX-repo-url>
node designerX/bin/designerx.mjs install --providers=claude --dir /path/to/your/project
```

Other forms: `--providers=all` installs for every platform, `--global` installs into your home folder (hooks are not touched), `--no-hooks` skips hook files. `designerx list` shows all platforms; `designerx uninstall` removes everything DesignerX added.

Your first five minutes

Step	Command	Result
1	<code>/designerx init</code>	PRODUCT.md: who it is for, purpose, brand, principles
2	<code>/designerx document</code>	DESIGN.md: your real palette, type and components
3	<code>/designerx audit</code>	Severity-ranked technical report
4	<code>/designerx critique</code>	UX review with prioritized fixes
5	<code>/designerx polish</code>	Final pass before shipping

Invoking commands

Where	Syntax
Most agents (Claude Code, Cursor, Gemini, Copilot, ...)	<code>/designerx <command> [target]</code>
OpenAI Codex	<code>\$designerx <command> [target]</code>
Terminal	<code>designerx <verb> [options]</code> (or <code>npx designerx ...</code>)

Run `/designerx` with no argument for a context-aware menu that recommends what to do next based on your project's state.

2. Command reference

Twenty-seven commands in seven groups. Commands marked **Original** were written for DesignerX and do not exist upstream.

Command	Group	Purpose
craft	Build	Deprecated alias for an ordinary new-work request.
shape	Build	Plan UX/UI before writing code.
init	Build	Capture durable product context in PRODUCT.md.
document	Build	Generate DESIGN.md from existing project code.
extract	Build	Pull reusable tokens and components into a design system.
critique	Evaluate	UX design review with heuristic scoring.
audit	Evaluate	Technical quality checks: accessibility, performance, responsiveness.
polish	Refine	Final quality pass before shipping.
bolder	Refine	Amplify safe or bland designs.
quieter	Refine	Tone down aggressive or overstimulating designs.
distill	Refine	Strip to essence and remove complexity.
harden	Refine	Make it production-ready: errors, i18n, edge cases.
onboard	Refine	Design first-run flows, empty states and activation.
animate	Enhance	Add purposeful animation and motion.
colorize	Enhance	Add strategic color to monochromatic UIs.
typeset	Enhance	Improve typography hierarchy and fonts.
layout	Enhance	Fix spacing, rhythm and visual hierarchy.
delight	Enhance	Add personality and memorable touches.
overdrive	Enhance	Push past conventional limits.
signature	Enhance	DesignerX original. Define one ownable visual signature. (Original)
clarify	Fix	Improve UX copy, labels and error messages.
adapt	Fix	Adapt for different devices and screen sizes.
optimize	Fix	Diagnose and fix UI performance.
localize	Fix	DesignerX original. Adapt copy, formats and layout to another locale. (Original)
live	Iterate	Visual variant mode in the browser.
generate	Iterate	N variants of a named element, no manual picking.
handoff	Ship	DesignerX original. Write a developer handoff spec. (Original)

Build

craft [feature]

Deprecated alias for an ordinary new-work request.

Kept for muscle memory. It routes to the same new-work flow as simply describing the feature you want, with design context loaded first.

```
/designerx craft pricing page for a B2B invoicing tool
```

Tip: Prefer plain requests or `shape` followed by implementation.

shape [feature]

Plan UX/UI before writing code.

Runs a short discovery, then produces a design brief: users, goals, content, structure, states and constraints. No code is written until you confirm the brief.

```
/designerx shape onboarding for a habit-tracking app
```

Tip: Use before big features. A confirmed brief makes every later command sharper.

init

*Capture durable product context in **PRODUCT.md**.*

Interviews you once about users, purpose, brand and principles and writes **PRODUCT.md**. Every other command reads it, so output stops sounding generic.

```
/designerx init
```

Tip: Run it first in any new project. Re-run when the product direction changes.

document

*Generate **DESIGN.md** from existing project code.*

Reads your real CSS, tokens and components and writes **DESIGN.md** (palette, type, spacing, components) so the design system is written down.

```
/designerx document
```

Tip: Run after `init`; re-run after large refactors to keep it honest.

extract [target]

Pull reusable tokens and components into a design system.

Finds repeated colors, spacing values and UI patterns in the target and consolidates them into tokens and shared components.

```
/designerx extract src/components/checkout
```

Tip: Review the proposed token names before accepting the refactor.

Evaluate

critique [target]

UX design review with heuristic scoring.

Scores hierarchy, clarity, consistency, emotional fit and more, then lists prioritized fixes and recommends which DesignerX commands to run next.

```
/designerx critique the dashboard header
```

Tip: Use it as a second opinion before polishing.

audit [target]

Technical quality checks: accessibility, performance, responsiveness.

Measures contrast, keyboard paths, semantics, layout shifts, responsiveness and anti-patterns, and returns a severity-ranked report. Includes a native-app variant.

```
/designerx audit src/pages/Signup.tsx
```

Tip: Pair with the terminal detector in CI: `designerx detect src`.

Refine

polish [target]

Final quality pass before shipping.

Fixes alignment, spacing, consistency, states and micro-details so a good screen becomes a finished one.

```
/designerx polish the settings screen
```

Tip: Always last in the sequence, never first.

bolder [target]

Amplify safe or bland designs.

Increases contrast of scale, color and composition while keeping the product's identity and accessibility.

```
/designerx bolder landing hero
```

Tip: Follow with `quieter` on secondary areas to keep a calm baseline.

quieter [target]

Tone down aggressive or overstimulating designs.

Reduces visual noise: saturation, competing accents, heavy effects, so content leads.

```
/designerx quieter admin tables
```

Tip: Good for dense tools used for hours.

distill [target]

Strip to essence and remove complexity.

Removes decoration, redundant UI and low-value options until the primary task is obvious.

```
/designerx distill checkout form
```

Tip: Ask it what it removed so nothing important disappears silently.

harden [target]

Make it production-ready: errors, i18n, edge cases.

Handles empty, loading, error and overflow states, long text, missing data, offline and localization risks.

```
/designerx harden the upload flow
```

Tip: Run before `handoff`.

onboard [target]

Design first-run flows, empty states and activation.

Turns blank screens into guided first steps and shortens time to the first success.

```
/designerx onboard project dashboard
```

Tip: Needs a clear definition of the 'aha' moment in PRODUCT.md.

Enhance

animate [target]

Add purposeful animation and motion.

Adds transitions and feedback that explain state changes, with reduced-motion support.

```
/designerx animate modal open and close
```

Tip: Motion should explain, not decorate.

colorize [target]

Add strategic color to monochromatic UIs.

Introduces an accessible, restrained color system for state, emphasis and brand.

```
/designerx colorize the analytics view
```

Tip: Check results with `audit` for contrast.

typeset [target]

Improve typography hierarchy and fonts.

Chooses and pairs typefaces, fixes scale, weight, line length and rhythm.

```
/designerx typeset blog article template
```

Tip: Mention the languages you support so character coverage is checked.

layout [target]

Fix spacing, rhythm and visual hierarchy.

Rebuilds spacing scale, alignment and grouping so structure is visible at a glance.

```
/designerx layout pricing cards
```

Tip: Use before `polish` on messy screens.

delight [target]

Add personality and memorable touches.

Adds small moments of character (copy, micro-interactions, empty-state art) without hurting usability.

```
/designerx delight the success screen
```

Tip: Reserve it for moments of success, not for error paths.

overdrive [target]

Push past conventional limits.

Ambitious, high-effort treatment (advanced effects, unusual interaction models) for showcase surfaces.

```
/designerx overdrive the product launch page
```

Tip: Not for dense productivity UI.

signature [target] **ORIGINAL**

DesignerX original. Define one ownable visual signature.

Finds a recognizable move rooted in the product's real world (structure, type, color, motion or detail), writes it as an implementable rule and applies it with restraint. Verified by crop and removal tests.

```
/designerx signature the whole marketing site
```

Tip: One primary signature plus at most one echo.

Fix

clarify [target]

Improve UX copy, labels and error messages.

Rewrites unclear microcopy so every label, error and empty state says what happened and what to do.

```
/designerx clarify payment errors
```

Tip: Run `localize` afterwards if you ship in more than one language.

adapt [target]

Adapt for different devices and screen sizes.

Fixes breakpoints, touch targets, input modes and platform conventions. Includes a native-app variant.

```
/designerx adapt dashboard for mobile
```

Tip: State the smallest viewport you must support.

optimize [target]

Diagnose and fix UI performance.

Finds render, asset, font and layout-shift problems and fixes the highest impact ones first.

```
/designerx optimize homepage
```

Tip: Include a throttled profile if you have one.

localize [locale] [target] ORIGINAL

DesignerX original. Adapt copy, formats and layout to another locale.

Audits code for i18n blockers, rewrites copy idiomatically, applies Intl formats, plural/gender rules and RTL-safe logical CSS, then verifies with pseudo-localization. Contains pt-BR notes (você, R\$ 1.234,56, CPF/CNPJ/CEP, text expansion).

```
/designerx localize pt-BR checkout
```

Tip: If no locale is given it uses PRODUCT.md, otherwise asks once.

Iterate

live

Visual variant mode in the browser.

Starts a local session: pick an element in your running app, describe the change, and compare generated variants in place before accepting one.

```
/designerx live
```

Tip: Needs a running dev server. Not covered by this manual's automated tests.

generate [n] [action] [element]

N variants of a named element, no manual picking.

Like live mode but you name the element: it produces N alternatives to choose from.

```
/designerx generate 4 variations hero headline
```

Tip: Keep n between 3 and 6.

Ship

handoff [target] ORIGINAL

DesignerX original. Write a developer handoff spec.

Reads the built UI and writes HANDOFF.md: tokens, component states, breakpoints, content rules, accessibility, motion, open questions and an acceptance checklist. Every claim carries a file:line or measured value.

```
/designerx handoff src/features/billing
```

Tip: Run `harden` and `audit` first so gaps are real findings.

3. Terminal verbs

The launcher runs outside your agent, for scripts, CI and quick checks. Use `designerx` (npm) or the launcher inside the installed skill, for example `.claude/skills/designerx/scripts/designerx`.

Verb	What it does
<code>detect [file dir url ...]</code>	Scans UI code for anti-patterns and quality issues. Exit code 0 means clean, 2 means findings. Add <code>--json</code> for machine output.
<code>context</code>	Loads <code>PRODUCT.md</code> and <code>DESIGN.md</code> context for the current project (what agents read first).
<code>hooks on off status</code>	Enables, disables or inspects the automatic design detector that runs after your agent edits UI files.
<code>hooks</code> <code>ignore-rule ignore-file ignore-value reset</code>	Suppression ladder for false positives, from narrow to broad.
<code>doctor</code>	Reports and repairs drift in DesignerX project files.
<code>ignores</code>	Manages detector ignore rules.
<code>pin pin <command></code>	Creates a standalone shortcut, for example <code>/polish</code> , in every installed platform of the project.
<code>pin unpin <command></code>	Removes that shortcut (only if DesignerX created it).
<code>help, version</code>	Local help and version information.

CI example

```
# fail the build when UI anti-patterns are found
npx designerx detect src/ || exit 1
```

Inline suppression

In code you can mark intentional exceptions with the detector's comment directives (they keep the upstream spelling because the engine reads them), for example a `impeccable-disable-next-line` comment placed above the line. Prefer fixing the issue; use suppression for deliberate design choices.

Hooks

Installed hook files make supported agents run the detector after UI edits and feed findings back to the agent. Toggle per project with `designerx hooks off` and `on`. State lives in `.impeccable/config.json`.

Environment variables

Variable	Effect
DESIGNERX_HOME	Engine cache folder (default ~/.designerx).
DESIGNERX_BIN	Use an already installed engine binary; skips download.
DESIGNERX_DOWNLOAD_BASE	Mirror for engine downloads (needs matching .sha256 files).
DESIGNERX_ONLINE=1	Re-enable engine telemetry, update and staleness checks (off by default).
DESIGNERX_OFFLINE=1	Block the single hosted call (concept-seed roll). The engine falls back locally.
DESIGNERX_API_URL	Point that call at your own service.
DESIGNERX_RAW=1	Print engine output without the DesignerX naming layer.

4. Workflows

Goal	Sequence
New product	init → shape → (build) → document → audit → polish
Rescue an ugly screen	critique → layout → typeset → colorize → polish
Make it distinctive	document → signature → bolder or quieter → polish
Ship-ready	harden → audit → clarify → polish → handoff
Ship in Brazil	clarify → localize pt-BR → harden → audit
Explore options	live or generate 4 variations <element>
Weekly hygiene	detect in CI + hooks on + document after refactors

Good habits

1. Keep `PRODUCT.md` and `DESIGN.md` current; every command is better with them.
2. Give a target (a file, folder or screen name) instead of 'everything'.
3. Review diffs: DesignerX edits real code.
4. Run `polish` last and `audit` after any big change.

5. Network and privacy

The core engine is a compiled binary from the upstream project. On first use the launcher downloads it from the upstream release channel and verifies its SHA-256 checksum; it refuses to run unverified downloads. Telemetry, update checks and staleness checks are disabled by default. The only remaining hosted call is the concept-seed roll, which sends scope, mode, a short seed and a counter, and no project content. Set `DESIGNERX_OFFLINE=1` to block it.

6. Troubleshooting

Symptom	Fix
'cannot create cache directory' or download fails	Allow network once, or set <code>DESIGNERX_HOME</code> to a writable folder, or <code>DESIGNERX_BIN</code> to a preinstalled engine.
Command not found in the agent	Restart the agent session after install. Check the skill folder exists, for example <code>.claude/skills/designerx</code> .
'NO_PRODUCT_MD' message	Run <code>/designerx init</code> to create <code>PRODUCT.md</code> .
Hooks do nothing	Run <code>designerx hooks status</code> ; enable with <code>designerx hooks on</code> . Confirm the hook file for your platform was installed.
Windows: pin or help missing	Those verbs are implemented in the POSIX launcher. Use Git Bash or WSL, or the npm CLI.

7. Credits and license

DesignerX is crafted by **Daniel Rodrigues** and released under the Apache License 2.0. It is a derivative of **Impeccable** by **Paul Bakaus** (Copyright 2025), also Apache-2.0. The names and marks of the original project are not used under license. See `NOTICE.md` in the repository for the full list of changes and third-party attributions.